Assignment #1 – Lab Assignment

Data Structures (CSC211)

STUDENT NAME:

ABDULLAH HASEEB ABBASI

REGISTRATION NO:

SP25-BCS-060

TASK 1: DISPLAY SINGLY LINKED LIST IN REVERSE

Display the linked list from the last node to the first without changing the links.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void reverseDisplay(Node* head) {
    if (head == NULL) return;
    reverseDisplay(head->next);
    cout << head->data << " ";
}

int main() {
    Node* a = new Node{1, NULL};
    Node* b = new Node{2, NULL};
    Node* c = new Node{3, NULL};
    Node* d = new Node{4, NULL};
```

```
    a->next = b; b->next = c; c->next = d;

    reverseDisplay(a);
    return 0;
}
```

TASK 2: REVERSE SINGLY LINKED LIST

Reverse the linked list by changing only the links. The nodes themselves are not recreated.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* reverseList(Node* head) {
    Node* prev = NULL;
    Node* cur = head;

    while (cur != NULL) {
        Node* next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    return prev;
}

void display(Node* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* a = new Node{1, NULL};
    Node* b = new Node{2, NULL};
    Node* c = new Node{3, NULL};
    Node* d = new Node{4, NULL};
    a->next = b; b->next = c; c->next = d;

    a = reverseList(a);
    display(a);
    return 0;
}
```

TASK 3: REMOVE DUPLICATE CONTACTS

Remove all duplicate values so that every contact appears only once.

```
#include <iostream>
using namespace std;

struct Node {
```

```
    int data;
    Node* next;
};

void removeDuplicates(Node* head) {
    for (Node* p = head; p != NULL; p = p->next) {
        Node* q = p;
        while (q->next != NULL) {
            if (q->next->data == p->data) {
                Node* t = q->next;
                q->next = q->next->next;
                delete t;
            } else {
                q = q->next;
            }
        }
    }
}

void display(Node* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* a = new Node{10, NULL};
    Node* b = new Node{20, NULL};
    Node* c = new Node{10, NULL};
    Node* e = new Node{30, NULL};
    Node* f = new Node{20, NULL};
    a->next=b; b->next=c; c->next=e; e->next=f;

    removeDuplicates(a);
    display(a);
    return 0;
}
```

TASK 4: DETECT LOOP IN SINGLY LINKED LIST

Detect whether a loop exists using the slow and fast pointer method.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

bool hasLoop(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
```

```
    }
    return false;
}

int main() {
    Node* a = new Node{1, NULL};
    Node* b = new Node{2, NULL};
    Node* c = new Node{3, NULL};
    a->next=b; b->next=c; c->next=b;

    if (hasLoop(a)) cout << "Loop exists";
    else cout << "No loop";
    return 0;
}
```

TASK 5: FIND MIDDLE NODE

Find the middle node. For an even number of nodes, choose the left middle node.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* middle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL && fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

int main() {
    Node* a = new Node{1, NULL};
    Node* b = new Node{2, NULL};
    Node* c = new Node{3, NULL};
    Node* e = new Node{4, NULL};
    Node* f = new Node{5, NULL};
    a->next=b; b->next=c; c->next=e; e->next=f;

    cout << middle(a)->data;
    return 0;
}
```

TASK 6: SEPARATE EVEN AND ODD VALUES

Divide the original list into two linked lists: one for even values and one for odd values.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
```

```
    Node* next;
};

void add(Node*& head, Node*& tail, int x) {
    Node* n = new Node{x, NULL};
    if (head == NULL) head = tail = n;
    else {
        tail->next = n;
        tail = n;
    }
}

void split(Node* head, Node*& even, Node*& odd) {
    Node *et = NULL, *ot = NULL;
    while (head != NULL) {
        if (head->data % 2 == 0) add(even, et, head->data);
        else add(odd, ot, head->data);
        head = head->next;
    }
}

void display(Node* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* a = new Node{1, NULL};
    Node* b = new Node{2, NULL};
    Node* c = new Node{3, NULL};
    Node* e = new Node{4, NULL};
    a->next=b; b->next=c; c->next=e;

    Node *even=NULL, *odd=NULL;
    split(a, even, odd);

    cout << "Even: "; display(even);
    cout << "\nOdd: "; display(odd);
    return 0;
}
```

TASK 7: REVERSE FIRST AND SECOND HALF SEPARATELY

Reverse each half of the list separately. For 1 to 8, the result is 4 3 2 1 8 7 6 5.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* reversePart(Node* head, Node* stop) {
    Node* prev = NULL;
    Node* cur = head;
    while (cur != stop) {
```

```
        Node* n = cur->next;
        cur->next = prev;
        prev = cur;
        cur = n;
    }
    return prev;
}

void solve(Node*& head, int n) {
    int half = n / 2;
    Node* second = head;
    for (int i = 0; i < half; i++) second = second->next;

    Node* first = reversePart(head, second);
    Node* secondHead = reversePart(second, NULL);

    Node* tail = first;
    while (tail->next != NULL) tail = tail->next;
    tail->next = secondHead;
    head = first;
}

void display(Node* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* head = NULL;
    Node* tail = NULL;
    for (int i = 1; i <= 8; i++) {
        Node* n = new Node{i, NULL};
        if (head == NULL) head = tail = n;
        else { tail->next=n; tail=n; }
    }

    solve(head, 8);
    display(head);
    return 0;
}
```

TASK 8: REMOVE DUPLICATE STAMPS

Remove duplicate designs from the linked list, keeping the first occurrence.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void removeDuplicates(Node* head) {
    Node* p = head;
    while (p != NULL) {
        Node* q = p;
```

```
        while (q->next != NULL) {
            if (q->next->data == p->data) {
                Node* t = q->next;
                q->next = q->next->next;
                delete t;
            } else q = q->next;
        }
        p = p->next;
    }
}

int main() {
    Node* a = new Node{5, NULL};
    Node* b = new Node{8, NULL};
    Node* c = new Node{5, NULL};
    Node* e = new Node{8, NULL};
    a->next=b; b->next=c; c->next=e;

    removeDuplicates(a);

    while (a != NULL) {
        cout << a->data << " ";
        a = a->next;
    }
    return 0;
}
```

TASK 9: DELETE ALL OCCURRENCES OF A GIVEN VALUE

Delete every node having the specified identifier, including occurrences at the beginning, middle and end.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void deleteValue(Node*& head, int key) {
    while (head != NULL && head->data == key) {
        Node* t = head;
        head = head->next;
        delete t;
    }

    Node* p = head;
    while (p != NULL && p->next != NULL) {
        if (p->next->data == key) {
            Node* t = p->next;
            p->next = p->next->next;
            delete t;
        } else p = p->next;
    }
}

void display(Node* head) {
    while (head != NULL) {
```

```
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* a = new Node{7, NULL};
    Node* b = new Node{5, NULL};
    Node* c = new Node{7, NULL};
    Node* e = new Node{9, NULL};
    Node* f = new Node{7, NULL};
    a->next=b; b->next=c; c->next=e; e->next=f;

    deleteValue(a, 7);
    display(a);
    return 0;
}
```

TASK 10: SWAP NODES IN PAIRS

Swap every two consecutive nodes by changing links. For 1 2 3 4 5 6, the result is 2 1 4 3 6 5.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void swapPairs(Node*& head) {
    Node dummy{0, head};
    Node* prev = &dummy;

    while (prev->next != NULL && prev->next->next != NULL) {
        Node* a = prev->next;
        Node* b = a->next;

        a->next = b->next;
        b->next = a;
        prev->next = b;

        prev = a;
    }
    head = dummy.next;
}

int main() {
    Node* head = NULL;
    Node* tail = NULL;
    for (int i = 1; i <= 6; i++) {
        Node* n = new Node{i, NULL};
        if (head == NULL) head = tail = n;
        else { tail->next=n; tail=n; }
    }

    swapPairs(head);

    while (head != NULL) {
```

```
        cout << head->data << " ";
        head = head->next;
    }
    return 0;
}
```

Tasks of Doubly Linked List

TASK 1: REVERSE DOUBLY LINKED LIST BY SWAPPING DATA

Swap the first value with the last, the second with the second-last, and continue to the center.

```
#include <iostream>
using namespace std;

struct Node {
    string data;
    Node* prev;
    Node* next;
};

void reverseData(Node* head, Node* tail) {
    while (head != tail && head->prev != tail) {
        string x = head->data;
        head->data = tail->data;
        tail->data = x;
        head = head->next;
        tail = tail->prev;
    }
}

void display(Node* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* a=new Node{"Alice",NULL,NULL};
    Node* b=new Node{"Bob",NULL,NULL};
    Node* c=new Node{"Charlie",NULL,NULL};
    Node* e=new Node{"Dana",NULL,NULL};
    Node* f=new Node{"Eva",NULL,NULL};
    Node* g=new Node{"Frank",NULL,NULL};

    a->next=b; b->prev=a; b->next=c; c->prev=b;
    c->next=e; e->prev=c; e->next=f; f->prev=e;
    f->next=g; g->prev=f;

    reverseData(a,g);
    display(a);
    return 0;
}
```

TASK 2: DOUBLY LINKED PLAYLIST

The statement describes forward and backward playlist navigation. The following program supports adding songs, displaying forward, and displaying backward.

```
#include <iostream>
using namespace std;

struct Node {
    string song;
    Node* prev;
    Node* next;
};

void add(Node*& head, Node*& tail, string s) {
    Node* n = new Node{s, tail, NULL};
    if (head == NULL) head = tail = n;
    else {
        tail->next = n;
        tail = n;
    }
}

void forward(Node* head) {
    while (head != NULL) {
        cout << head->song << " ";
        head = head->next;
    }
}

void backward(Node* tail) {
    while (tail != NULL) {
        cout << tail->song << " ";
        tail = tail->prev;
    }
}

int main() {
    Node *head=NULL, *tail=NULL;
    add(head,tail,"Song1");
    add(head,tail,"Song2");
    add(head,tail,"Song3");

    cout << "Forward: ";
    forward(head);
    cout << "\nBackward: ";
    backward(tail);
    return 0;
}
```

Task 3: Hierarchical Store Inventory

Use linked structures for stores, sections, and items. The methods add a section, add/remove an item, display a section, and display a store.

```
#include <iostream>
using namespace std;

struct Item {
```

```
    string name;
    Item* next;
};

struct Section {
    string name;
    Item* items;
    Section* next;
};

struct Store {
    string name;
    Section* sections;
    Store* next;
};

void addStore(Store*& stores, string name) {
    Store* n = new Store{name, NULL, stores};
    stores = n;
}

Store* findStore(Store* stores, string name) {
    while (stores != NULL) {
        if (stores->name == name) return stores;
        stores = stores->next;
    }
    return NULL;
}

Section* findSection(Store* s, string name) {
    Section* p = s->sections;
    while (p != NULL) {
        if (p->name == name) return p;
        p = p->next;
    }
    return NULL;
}

void addSection(Store* s, string name) {
    Section* n = new Section{name, NULL, s->sections};
    s->sections = n;
}

void addItem(Store* s, string sec, string item) {
    Section* x = findSection(s, sec);
    if (x == NULL) return;
    Item* n = new Item{item, x->items};
    x->items = n;
}

void removeItem(Store* s, string sec, string item) {
    Section* x = findSection(s, sec);
    if (x == NULL) return;

    while (x->items != NULL && x->items->name == item) {
        Item* t = x->items;
        x->items = x->items->next;
        delete t;
    }
```

```
    Item* p = x->items;
    while (p != NULL && p->next != NULL) {
        if (p->next->name == item) {
            Item* t = p->next;
            p->next = p->next->next;
            delete t;
        } else p = p->next;
    }
}

void displaySection(Store* s, string sec) {
    Section* x = findSection(s, sec);
    if (x == NULL) return;
    cout << x->name << ": ";
    Item* p = x->items;
    while (p != NULL) {
        cout << p->name << " ";
        p = p->next;
    }
}

void displayStore(Store* s) {
    Section* x = s->sections;
    while (x != NULL) {
        displaySection(s, x->name);
        cout << "\n";
        x = x->next;
    }
}

int main() {
    Store* stores = NULL;
    addStore(stores, "Islamabad");

    Store* s = findStore(stores, "Islamabad");
    addSection(s, "Grocery");
    addSection(s, "Toys");
    addItem(s, "Grocery", "Milk");
    addItem(s, "Grocery", "Bread");
    addItem(s, "Toys", "Car");

    displayStore(s);
    return 0;
}
```

Task 4: Special Seating Pattern

Keep the first and last seats fixed and swap the inner seats from both ends toward the center to produce 1 8 3 6 5 4 7 2 9.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};
```

```
void pattern(Node* head, Node* tail, int n) {
    if (n < 3) return;

    Node* left = head->next;
    Node* right = tail->prev;

    while (left != right && left->next != right) {
        int x = left->data;
        left->data = right->data;
        right->data = x;

        left = left->next->next;
        right = right->prev->prev;
    }
}

int main() {
    Node* head=NULL; Node* tail=NULL; Node* prev=NULL;

    for (int i=1; i<=9; i++) {
        Node* n = new Node{i, prev, NULL};
        if (head == NULL) head=n;
        else prev->next=n;
        prev=n;
        tail=n;
    }

    pattern(head, tail, 9);

    while (head != NULL) {
        cout << head->data << " ";
        head=head->next;
    }
    return 0;
}
```

Tasks of Circular Linked List

Josephus Problem

Given N persons and M, skip M-1 persons and remove the Mth person until one survivor remains. Find the safe position.

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

int josephus(int n, int m) {
    Node* head = NULL;
    Node* tail = NULL;

    for (int i = 1; i <= n; i++) {
        Node* x = new Node{i, NULL};
        if (head == NULL) head = tail = x;
        else { tail->next=x; tail=x; }
    }
    tail->next = head;

    Node* prev = tail;
    Node* cur = head;

    while (cur->next != cur) {
        for (int i=1; i<m; i++) {
            prev=cur;
            cur=cur->next;
        }

        prev->next=cur->next;
        delete cur;
        cur=prev->next;
    }

    int ans=cur->data;
    delete cur;
    return ans;
}

int main() {
    int n, m;
    cout << "N: ";
    cin >> n;
    cout << "M: ";
    cin >> m;

    cout << "Safe position: " << josephus(n,m);
    return 0;
}
```

Circular Task Scheduling System

Maintain tasks in a circular linked list with add, remove, get-next, display, and status-update operations.

```
#include <iostream>
using namespace std;

struct Node {
    string name;
    int priority;
    string status;
    Node* next;
};

class TaskList {
    Node* head;
    Node* current;

public:
    TaskList() {
        head = NULL;
        current = NULL;
    }

    void addTask(string name, int priority, string status) {
        Node* n = new Node{name, priority, status, NULL};

        if (head == NULL) {
            head = n;
            n->next = n;
            current = n;
            return;
        }

        Node* p = head;
        while (p->next != head) p = p->next;
        p->next = n;
        n->next = head;
    }

    void removeTask(string name) {
        if (head == NULL) return;

        Node* prev = head;
        while (prev->next != head && prev->next->name != name)
            prev = prev->next;

        Node* del = NULL;
        if (head->name == name) {
            if (head->next == head) {
                delete head;
                head = current = NULL;
                return;
            }

            Node* tail = head;
            while (tail->next != head) tail = tail->next;
            del = head;
```

```
            head = head->next;
            tail->next = head;
        } else if (prev->next->name == name) {
            del = prev->next;
            prev->next = del->next;
        }

        if (del != NULL) {
            if (current == del) current = del->next;
            delete del;
        }
    }

    void getNextTask() {
        if (head == NULL) return;

        if (current == NULL) current = head;
        Node* start = current;

        do {
            if (current->status == "pending") {
                cout << current->name << " ";
                current = current->next;
                return;
            }
            current = current->next;
        } while (current != start);

        cout << "No pending task";
    }

    void display() {
        if (head == NULL) return;

        Node* p = head;
        do {
            cout << p->name << " " << p->priority << " " << p->status << "\n";
            p = p->next;
        } while (p != head);
    }

    void updateStatus(string name, string status) {
        if (head == NULL) return;

        Node* p = head;
        do {
            if (p->name == name) {
                p->status = status;
                cout << p->name << " " << p->status;
                return;
            }
            p = p->next;
        } while (p != head);
    }
};

int main() {
    TaskList tasks;
```

```
    tasks.addTask("Task1", 1, "pending");
    tasks.addTask("Task2", 2, "completed");
    tasks.addTask("Task3", 1, "pending");

    tasks.display();
    cout << "\nNext: ";
    tasks.getNextTask();

    cout << "\nUpdated: ";
    tasks.updateStatus("Task1", "completed");

    cout << "\nRemove Task2\n";
    tasks.removeTask("Task2");
    tasks.display();

    return 0;
}
```